CSED233 · 데이터구조 · 포스텍

# Ch7. Priority Queue & Heap

## 1. Priority Queue (PQ): 동기

### Queue에 우선순위 추가

- **일반 Queue(큐)**: FIFO — 들어온 순서대로 나감
- **Priority Queue(우선순위 큐)**: 각 원소에 **priority(key)** 부여 → priority 순으로 나감
- Priority는 **unique일 필요 없음**
- 핵심 연산:
  - $\mathrm{Insert}(x, H)$: 임의 원소 삽입
  - $\mathrm{DeleteMin}(H)$: 최우선 원소 꺼내기 = $\mathrm{Min}(H) + \mathrm{Delete}$

Priority Queue(PQ)는 각 원소에 우선순위 키(priority key)를 부여하고, 삽입 순서가 아니라 키의 우선순위에 따라 원소를 처리하는 추상 자료형(ADT)이다. 이 장에서 다루는 Min-Priority Queue는 키가 작을수록 우선순위가 높으며, DeleteMin은 현재 저장된 원소 중 키가 최소인 원소 하나를 제거하고 반환한다. 우선순위 키는 서로 달라도 되고 같아도 된다. 다만 동일한 키를 가진 원소들의 처리 순서는 별도의 tie-breaking 규칙을 정의하지 않는 한 보장되지 않으며, 일반적인 binary heap은 동일 우선순위 원소에 대한 FIFO 순서를 보장하지 않는다.

### 왜 필요한가

- 응급실 환자 분류, OS 프로세스 스케줄링, Dijkstra 최단경로, 이벤트 시뮬레이션 등
- 처리 순서가 도착 순서에 의해 결정되는 작업에는 FIFO Queue가 적합하지만, 응급실의 중증도 분류, 최단경로 탐색, 이벤트 시뮬레이션처럼 작업마다 중요도나 처리 시점이 다른 경우에는 Priority Queue가 더 적합하다. 따라서 자료구조의 선택은 처리 순서에 대한 요구사항에 따라 달라진다.

## 2. 단순 구현의 한계

### 3가지 naive 구현 비교

두 가지 단순 연결 리스트 구현과 Binary Heap을 비교한다. Unsorted linked list는 삽입이 빠르지만 최솟값 삭제가 느리고, Sorted linked list는 최솟값 삭제가 빠르지만 삽입이 느리다. Binary Heap은 완전 이진 트리와 heap-order property를 이용해 두 연산 모두 최악의 경우 $O(\log n)$ 시간에 수행한다.

| 구현 | Insert | DeleteMin | 이유 |
|---|---|---|---|
| Unsorted linked list | $O(1)$ | $O(n)$ | 삽입은 head, 삭제는 전체 스캔 |
| Sorted linked list | $O(n)$ | $O(1)$ | 삽입 위치 탐색, 삭제는 head |
| **Heap(힙)** | $O(\log n)$ | $O(\log n)$ | **균형** |

### Insight

- Unsorted/Sorted 모두 **한 쪽은** $O(n)$
- → **둘 다** $O(\log n)$ 달성이 Heap의 의미
- Unsorted linked list는 Insert가 $O(1)$, DeleteMin이 $O(n)$이고, Sorted linked list는 Insert가 $O(n)$, DeleteMin이 $O(1)$이다. 반면 Binary Heap은 두 연산을 모두 최악의 경우 $O(\log n)$에 수행하므로, 큐에 많은 원소가 유지되고 Insert와 DeleteMin이 모두 빈번한 워크로드에서 두 단순 연결 리스트 구현보다 유리하다. 다만 Heap이 모든 워크로드와 모든 자료구조에 대해 최적이라는 뜻은 아니며, 실제 성능은 연산의 빈도, 현재 원소 수, 요구되는 추가 연산에 따라 달라진다.

## 3. Binary Heap: 정의

### 두 가지 Property

**Min-Heap**은 다음 두 조건을 만족하는 tree:

1. **Structural property: Complete Binary Tree (CBT)**
   - → array로 pointer 없이 저장 가능 (Ch5 numbering 활용)
2. **Heap-order property: Min-tree**
   - $p(\text{parent}) \leq p(\text{child})$ for all nodes

- → root가 전체 최솟값

> **Max-Heap**: (2')만 부등호 반대 $p(\text{parent}) \geq p(\text{child})$

Heap-order property (Min-Heap): $p(v)$를 노드 $v$에 저장된 원소의 우선순위 값(priority key)이라고 정의한다. Min-Heap에서는 트리에 존재하는 모든 부모-자식 관계 $(u, v)$에 대해 부모 노드 $u$의 우선순위 값이 자식 노드 $v$의 우선순위 값보다 작거나 같아야 한다. $p(u) \leq p(v)$ for every parent-child pair $(u,v)$. 따라서 루트에서 임의의 노드까지 이어지는 경로를 따라 우선순위 값이 감소하지 않으며, Heap이 비어 있지 않다면 루트는 전체 노드 중 최소 우선순위 값을 가진다. 서로 다른 노드의 우선순위 값은 같을 수 있으므로 루트의 최솟값이 유일할 필요는 없다. Max-Heap에서는 반대로 모든 부모-자식 관계에 대해 $p(u) \geq p(v)$가 성립하며, 루트가 전체 최댓값을 가진다.

Structural property — Complete Binary Tree(CBT): 완전 이진 트리는 마지막 레벨을 제외한 모든 레벨이 가득 차 있고, 마지막 레벨의 노드들은 왼쪽부터 빈자리 없이 채워져 있는 이진 트리이다. 노드 수가 $n \geq 1$일 때 루트의 레벨을 0으로 정의하면 트리의 높이는 $h = \lfloor \log_2 n \rfloor$이다. CBT를 레벨 순서대로 1-based 배열 $A[1 \cdots n]$에 저장하면, 인덱스 $i > 1$인 노드의 부모 인덱스는 $\lfloor i/2 \rfloor$이고, 왼쪽 자식과 오른쪽 자식의 인덱스는 각각 $2i$, $2i+1$이다. 자식 인덱스가 $n$을 초과하면 해당 자식은 존재하지 않는다. 따라서 별도의 포인터 없이도 부모와 자식을 계산할 수 있으며, 인접한 배열 공간을 사용하므로 일반적으로 메모리 지역성에도 유리하다.

**CBT(완전 이진 트리) 선택 이유**

- Binary Heap의 Insert와 DeleteMin은 각각 한 노드를 부모 방향 또는 자식 방향으로 이동시키므로, 최악의 이동 횟수는 트리 높이 $h = O(\log n)$에 비례한다. CBT는 $n$개 노드에 대해 최소 높이를 보장하므로 두 연산을 효율적으로 수행할 수 있다. 반면 Find-Min은 루트 값을 읽기만 하므로 $O(1)$이고, Bottom-up Heapify는 여러 서브트리의 이동 비용을 합산하여 전체 $O(n)$ 시간이 걸린다.
- CBT는 $n$개 노드로 최소 높이 $\lfloor \log_2 n \rfloor$ 보장
- 또한 array 저장으로 pointer 오버헤드 제거 (cache friendly)

**Heap ≠ BST(이진 탐색 트리)**

- Heap-order property는 각 부모와 자식 사이의 크기 관계만 제한하며, 같은 레벨의 노드나 왼쪽 자식과 오른쪽 자식 사이의 크기 순서는 제한하지 않는다. 따라서 Min-Heap에서 왼쪽 자식이 반드시 오른쪽 자식보다 작아야 하는 것은 아니다. 다만 Complete Binary Tree의 구조를 유지하기 위해 왼쪽과 오른쪽의 위치 구분은 필요하며, 마지막 레벨은 반드시 왼쪽부터 채워야 한다.

- Binary Search Tree(BST)는 중복 키가 없는 경우 모든 노드에 대해 왼쪽 서브트리의 모든 키가 현재 노드의 키보다 작고, 오른쪽 서브트리의 모든 키가 현재 노드의 키보다 커야 한다. 중복 키를 허용하려면 별도의 중복 처리 규칙을 정의해야 한다. 반면 Min-Heap은 모든 부모-자식 관계에서 key(parent)≤key(child)만 만족하면 되고, 형제 노드나 서로 다른 서브트리 사이의 전체 정렬 순서는 요구하지 않는다. 따라서 BST는 탐색을 위한 전역적인 서브트리 순서 조건을 가지지만, Heap은 최솟값을 루트에 유지하기 위한 국소적인 순서 조건을 가진다.

## 4. Insert: Up-Heap Bubbling

---

**2단계 알고리즘**

1. **Hole 생성**: CBT의 structural property를 유지하기 위해 새 노드는 레벨 순서에서 다음으로 비어 있는 위치에 추가한다. 현재 마지막 레벨에 빈자리가 있다면 해당 레벨의 가장 왼쪽 빈자리를 채우고, 마지막 레벨이 가득 찼다면 새로운 레벨의 가장 왼쪽 위치에 추가한다. CBT를 레벨 순서로 저장하는 배열에서는 이 위치가 항상 배열의 다음 인덱스이므로, 새 원소를 배열 끝에 append하면 구조 조건이 자동으로 유지된다.
2. **Up-Heap Bubbling**: 배열 끝에 새 원소를 삽입한 뒤, 현재 노드의 키를 부모 노드의 키와 비교한다. 현재 노드가 루트가 아니고 key(current)<key(parent)이면 두 원소를 교환하고 부모 위치에서 같은 과정을 반복한다. 현재 노드가 루트에 도달하거나 key(parent)≤key(current)가 되면 교환을 중단한다. 이 과정은 CBT의 구조를 변경하지 않고 heap-order property만 복구한다.

예시: [2,5,3,9,7,8]에 1 삽입 → [2,5,3,9,7,8,1] → 부모 3과 교환 [2,5,1,9,7,8,3] → 부모 2와 교환 [1,5,2,9,7,8,3]. 최대 이동 횟수는 트리 높이 이하라 Insert 최악 $O(\log n)$.

**시간복잡도 유도**

- Bubbling은 **tree 높이만큼만** 위로 올라감
- CBT 높이 $= O(\log n)$
- 각 swap은 $O(1)$
- **→ Insert =** $O(\log n)$

## 5. DeleteMin: Down-Heap Bubbling

**2단계 알고리즘**

1. **Root 제거 및 마지막 원소 이동**: DeleteMin은 Min-Heap의 루트에 저장된 최솟값을 반환하고 제거하는 연산이다. Heap이 비어 있다면 삭제할 원소가 없으므로 정의된 예외 처리를 수행한다. 원소가 하나뿐이라면 루트를 제거하고 빈 Heap을 반환한다. 원소가 두 개 이상이라면 먼저 루트의 최솟값을 저장한 다음, 배열의 마지막 원소를 루트 위치로 이동시키고 배열의 마지막 칸을 제거한다. 이렇게 하면 CBT의 structural property가 유지된다.
2. **Down-Heap Bubbling**: 루트로 이동한 원소를 자식들과 비교한다. 두 자식이 모두 존재한다면 키가 더 작은 자식을 선택하고, 한 자식만 존재한다면 그 자식을 선택한다. 현재 노드의 키가 선택된 자식의 키보다 크면 두 원소를 교환하고, 교환된 위치에서 같은 과정을 반복한다. 현재 노드의 키가 선택된 자식의 키보다 작거나 같거나, 현재 노드가 리프라면 교환을 중단한다. 두 자식의 키가 같은 경우에는 어느 쪽을 선택해도 된다.

예시: [1,5,2,9,7,8,3] DeleteMin → 1 저장, 마지막 3을 루트로 [3,5,2,9,7,8] → 작은 자식 2와 교환 [2,5,3,9,7,8]. 최악 $O(\log n)$.

**왜 '더 작은' 자식과 비교?**

- 새 parent 자리에 오는 값이 **두 자식 중 작은 쪽**이어야 heap-order 유지
- 큰 자식과 swap하면 다른 쪽 자식 조건 위반
- → **반드시 min(left, right)과 비교** 후 swap

**시간복잡도**

- 트리 높이만큼 내려감 → $O(\log n)$

## 6. Quiz 정리

**Q1. Min-heap의 heap-order property의 정확한 정의는?**

> 힌트

Heap의 조건은 structural(모양)과 heap-order(값 관계) 두 가지다. 어느 쪽을 묻는지 구분하자.

> 정답

개념은 힌트 참고.

- A. **Every parent node has value ≤ its children** ✓
- B. 모든 레벨이 꽉 차 있음 → X. 이는 Perfect Binary Tree의 특징이지 Min-Heap의 heap-order property가 아니다. Perfect Binary Tree는 모든 레벨이 가득 찬 이진 트리이고, Full Binary Tree는 모든 노드가 자식을 0개 또는 2개 갖는 이진 트리이다. Complete Binary Tree는 마지막 레벨을 제외한 모든 레벨이 가득 차고 마지막 레벨은 왼쪽부터 채워진 트리이다. Min-Heap의 heap-order property는 트리의 모양이 아니라 모든 부모-자식 관계에서 부모의 키가 자식의 키보다 작거나 같다는 조건이다.
- C. Root가 최댓값 → X (min-heap은 root가 최솟값)
- D. Insertion은 root에서 → X (insertion은 맨 끝에서)

**정답: A — Every parent node has value ≤ its children.**

### Q2. 왜 $O(\log n)$이 naive들보다 나은가?

> **힌트**

Insert와 DeleteMin이 번갈아 빈번하게 일어나는 워크로드를 생각하자.

> **정답**

개념은 힌트 참고.

- Binary Heap은 Insert와 DeleteMin을 각각 최악의 경우 $O(\log n)$에 처리하지만, Unsorted linked list와 Sorted linked list는 두 연산 중 하나에 최악의 경우 $O(n)$ 시간이 필요하기 때문이다.
- 워크로드 예시 ①: $n$개 삽입 후 $n$번 DeleteMin — heap 전체 $O(n \log n)$, unsorted·sorted 모두 전체 $O(n^2)$
- 워크로드 예시 ②: 빈 PQ에서 Insert/DeleteMin을 번갈아 $n$회 — 저장 원소 최대 1개라 세 구현 모두 총 $O(n)$
- Heap의 장점은 모든 패턴에서 항상 더 빠르다는 것이 아니라, 많은 원소가 저장된 상태에서도 두 연산의 최악 시간복잡도를 모두 $O(\log n)$으로 유지한다는 것이다.

## 7. Heap Building: $n$개 원소로 heap 만들기

**두 방법**

**방법 1** (Obvious): 빈 heap에 하나씩 insert

- $\sum_{i=1}^{n} O(\log i) = O(n \log n)$

  **방법 2** (Heapify): 임의 순서로 CBT 만든 뒤 한 번에 정리

- $O(n)$ (증명은 § 8)

### Heapify 알고리즘 (Bottom-up)

```
입력: array-based CBT (n 노드)
1) k = floor(n/2)  ← 마지막 non-terminal node index
2) for k downto 1:
     if subtree(k)가 heap order 위반:
        down-heap bubble at k
```

- **핵심**: **밑에서 위로**, 각 서브트리를 heap으로 만들면서 올라감
- 이미 자식이 heap인 상태에서 root 하나만 내려보내면 됨 → down-heap 활용

### 왜 마지막 non-leaf 부터?

- **Leaf 노드는 자기 자신이 이미 heap** (자식 없음) → 건드릴 필요 없음
- 마지막 non-leaf = 가장 큰 index의 parent = $\lfloor n/2 \rfloor$
- Top-down Build는 빈 Heap에 원소를 하나씩 삽입하므로 각 삽입에서 최대 $O(\log n)$번의 이동이 발생할 수 있고, $n$번의 삽입을 합한 최악 시간복잡도는 $O(n \log n)$이다. 반면 Bottom-up Heapify에서는 리프에 가까운 많은 노드가 거의 이동하지 않고, 큰 이동 비용이 발생할 수 있는 노드는 상대적으로 적다. 따라서 전체 이동 비용의 합이 $O(n)$으로 제한된다. 이것은 Top-down에서 모든 개별 노드의 이동 거리가 반드시 더 길다는 뜻이 아니라, 두 알고리즘의 전체 최악 비용을 합산하는 방식이 다르다는 뜻이다.

## 8. Heapify Time Complexity: $O(n)$ 증명

### 핵심 아이디어

- Level 0 (root)는 높이 $h$, level $h$ (leaf)는 높이 0
- **Level $k$ 서브트리 개수** $\leq 2^k$
- **Level $k$ 서브트리 높이** $\leq h - k$
- Down-heap의 시간 = 서브트리 높이

**수식 유도**

$$\text{Total} \leq \sum_{k=0}^{h} 2^k \cdot (h-k)$$

치환 $j = h - k$:

$$= \sum_{j=0}^{h} 2^{h-j} \cdot j = 2^h \sum_{j=0}^{h} \frac{j}{2^j}$$

$\sum_{j=0}^{\infty} \frac{j}{2^j} = 2$ (등비수열 도함수 공식):

$$\leq 2^h \cdot 2 = 2^{h+1}$$

$n \approx 2^{h+1} - 1$ (정확히는 perfect tree의 등식; CBT는 $2^h \leq n < 2^{h+1}$을 만족하므로 $2^{h+1} \leq 2n$)
이므로:

$$\text{Total} = O(2^{h+1}) = O(n)$$

**교훈**

Obvious insert 방법 ($O(n \log n)$)과 다르게 heapify는 **선형 시간** ($O(n)$).
교훈: 루트를 Level 0으로 정의하면 Level 번호가 클수록 리프에 가까우며, CBT에서는 리프 근처의 노드가 루트 근처의 노드보다 훨씬 많다. 리프에 가까운 노드는 자신의 서브트리 높이가 작으므로 Down-Heap 과정에서 이동할 수 있는 최대 거리도 짧다. 반대로 루트 근처의 노드는 긴 거리를 이동할 수 있지만 그 수가 적다. 따라서 각 노드의 최대 이동 비용을 모두 합산해도 $O(n)$이 되며, 이것이 Bottom-up Heapify가 $O(n \log n)$이 아니라 $O(n)$ 시간에 수행되는 이유이다.

## Summary

| 연산 | 시간복잡도 | 방법 |
|---|---|---|
| Insert | $O(\log n)$ | Append → Up-heap bubble |

| | | |
|---|---|---|
| DeleteMin | $O(\log n)$ | Root 제거 → last 원소 이동 → Down-heap bubble |
| BuildHeap (one-by-one) | $O(n \log n)$ | $n$번 insert |
| **Heapify** (batch) | $O(n)$ | Bottom-up down-heap |
| Find-Min | $O(1)$ | Root 반환 |

- **Heap = CBT + heap-order** → array 저장, pointer 불필요
- Binary heap이 가장 흔함. Binomial/Fibonacci heap은 특수 상황